



Viatris — A Full Stack AI-Powered Healthcare Web Application

Frictionless OTP Appointment Booking · Digital Prescriptions · NGO Financial Aid Integration · Multi-Provider AI Health Assistant

Supervisor: Mr. Gaurav Negi (Asst. Professor, Grade-I) | Dhriti Sharma (221030092) · Raghav Singh Thakur (221030280) · Ujjwal Chauhan (221030417)

Department of Computer Science & Engineering — Jaypee University of Information Technology, Waknaghat

Group No. 70

Mid-Term
Evaluation
March 2026

INTRODUCTION

India's doctor-to-patient ratio of **1:1,457** places enormous administrative pressure on clinics. Despite growing digital infrastructure, fewer than **18%** of small clinics use any digital appointment system, with mandatory account registration being the primary barrier to adoption among elderly and low-literacy patients.

Viatris is a full-stack web application that eliminates registration friction by using the mobile phone number as the sole patient identity, verified through OTP — the same pattern Indian users already rely on daily for UPI payments and Aadhaar-linked systems.

The platform uniquely bridges three stakeholders — Patients, Doctors, and NGOs — in one coordinated system, backed by a 4-provider AI fallback chain ensuring near-100% chatbot availability.

USER ROLES & ACCESS



- Patient:** Book without registration, OTP verify, personal portal, upload/delete medical reports, NGO help requests, download prescriptions
- Doctor:** JWT-authenticated dashboard, manage appointments, write prescriptions, set availability slots, view analytics
- NGO:** View financial aid requests by category, update case status, add response notes, track outcomes

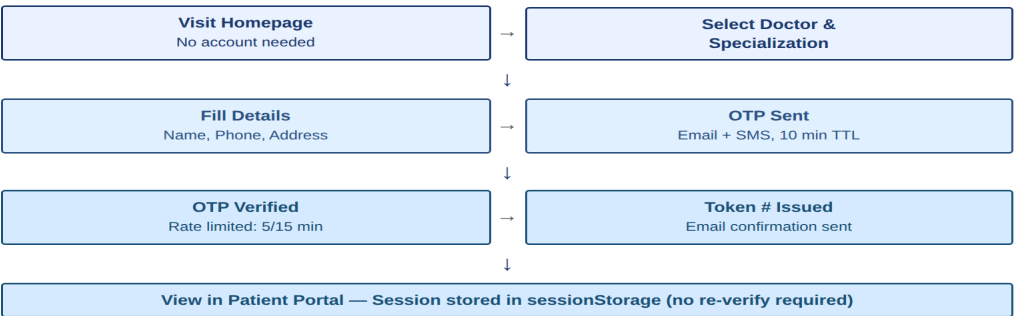
PROBLEM STATEMENT

- Mandatory email/password registration excludes elderly and low-literacy patients from digital healthcare booking
- Paper prescriptions are easily lost; no persistent digital record accessible across providers
- NGO financial assistance operates entirely separately from clinic systems, creating coordination gaps
- Many clinic platforms store passwords in plain text or weak MD5 hashing — a major security risk
- No existing platform combines no-registration booking, NGO integration, and AI assistance in one unified system

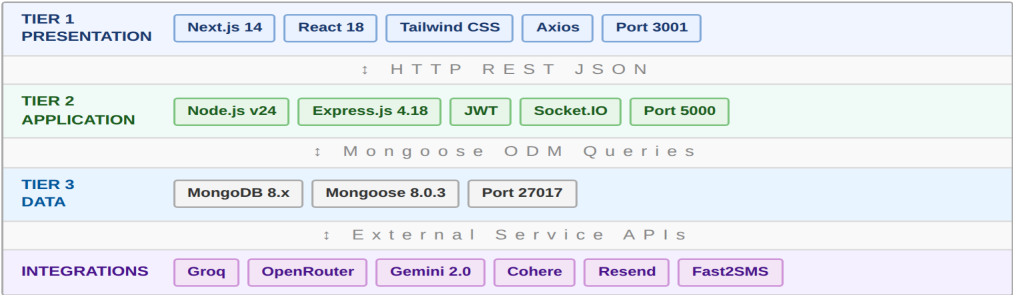
OBJECTIVES

- Implement no-registration OTP booking accessible to all literacy levels
- Build role-based secure dashboards for Doctors and NGOs with JWT authentication
- Create persistent digital prescription system with patient PDF download
- Integrate NGO financial assistance within the clinic appointment workflow
- Deploy multi-provider AI health assistant — Groq → OpenRouter → Gemini → Cohere
- Implement OWASP-compliant security: bcrypt, JWT, rate limiting, Helmet, CORS

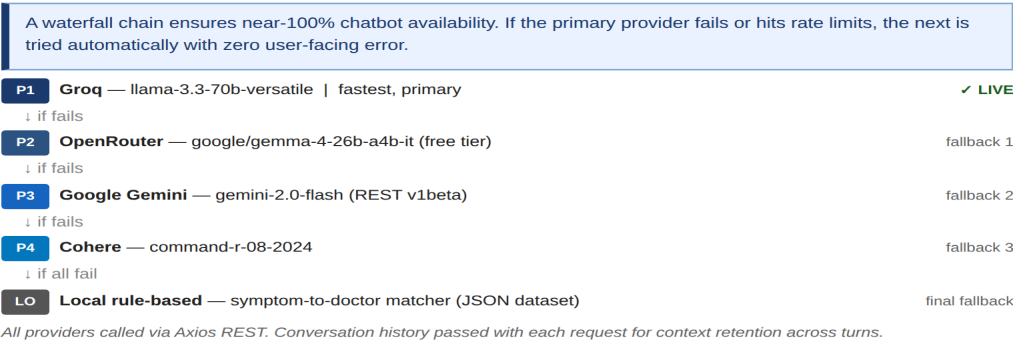
PATIENT BOOKING FLOW



SYSTEM ARCHITECTURE — 3-TIER DESIGN



AI CHATBOT — 4-PROVIDER FALLBACK CHAIN



All providers called via Axios REST. Conversation history passed with each request for context retention across turns.

TECHNOLOGY STACK

TECHNOLOGY	VERSION	LAYER	PURPOSE
Node.js	v24	Backend	Server runtime, non-blocking I/O
Express.js	4.18.2	Backend	REST routing + middleware chain
MongoDB	8.x	Backend	Flexible document storage, 6 collections
Mongoose	8.0.3	Backend	Schema, pre-save hooks, ODM
Next.js 14	14.0.4	Frontend	SSR + App Router file-based routing
React 18	18.2.0	Frontend	Hooks-based component UI
Tailwind CSS	3.3.6	Frontend	Utility-first responsive styling
bcryptjs	2.4.3	Security	Salted password hashing (10 rounds)
jsonwebtoken	9.0.3	Security	HS256 stateless JWT, 7-day expiry
express-rate-limit	8.4.1	Security	OTP brute force prevention: 5/15 min
Helmet.js	7.1.0	Security	11 HTTP security headers (CSP, HSTS)

KEY ALGORITHMS & IMPLEMENTATION

```
❶ bcrypt Password Hashing — Pre-Save Hook

// Auto-runs before every User.save() userSchema.pre("save", async function(next) { if
(!this.isModified("password")) return next(); const salt = await bcrypt.genSalt(10); this.password =
await bcrypt.hash(this.password, salt); next(); });

❷ JWT Auth Middleware — protect()

const protect = async (req, res, next) => { const token = req.headers.authorization?.split(" ")[1];
if (!token) return res.status(401).json({..}); const decoded = jwt.verify(token,
process.env.JWT_SECRET); req.user = await User.findById(decoded.id); next(); };

❸ Phone Normalisation for OTP Matching

// Matches +91 9560214848 OR 9560214848 const digits = phone .replace(/[\\s-\\(\\)]/g, "")
.replace(/^(\\+91|0091|91|0)/, ""); await Token.find({ "patient.phone": { $regex: digits, $options:
"i" } } );
```

EXPRESS.JS MIDDLEWARE CHAIN

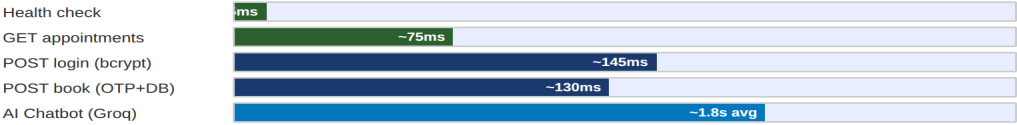
- cors() — validate origin whitelist (localhost 3000/3001)
- helmet() — 11 HTTP security headers applied automatically
- rateLimit() — 5 OTP attempts per 15 minutes per IP address
- protect() — JWT verify + attach user to req.user
- authorize() — role-based access check (patient / doctor / NGO / admin)
- Route Handler — business logic + JSON response returned

RESULTS — PERFORMANCE BENCHMARKS

Measured using Node.js perf_hooks, 50+ iterations, local dev machine + Postman + Supertest (Jest)

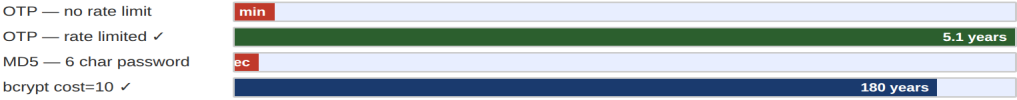
103 ms bcrypt hash (cost=10)	0.07 ms JWT verify per request	25 REST API endpoints	5154 Lines of source code
---	---	------------------------------------	--

API Endpoint Latency (estimated, local)



SECURITY ANALYSIS

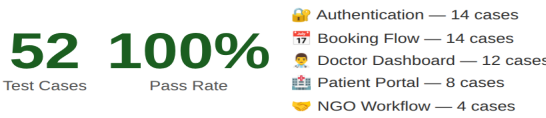
Brute Force Resistance (Calculated)



OWASP Top 10 Compliance

A01	Broken Access Control	✓ authChecked gate + router.replace
A02	Cryptographic Failures	✓ bcrypt + select:false on OTP
A05	Security Misconfiguration	✓ CORS whitelist + Helmet.js
A07	Auth & Session Failures	✓ Rate limit + sessionStorage TTL

TESTING RESULTS



TEST SCENARIO	INPUT	RESULT
Wrong OTP attempt	Incorrect 6-digit code	✓ 400 Invalid OTP
Direct URL access	Navigate /doctor/dashboard	✓ Redirect to /
OTP after 10 min	Expired OTP replay	✓ 400 OTP Expired
Brute force OTP	6th attempt in 15 min	✓ 429 Rate Limited
Password in DB	Query users collection	✓ bcrypt hash stored
Cross-role access	Doctor JWT → NGO route	✓ 403 Forbidden
Double booking	Same slot twice	✓ Slot lock enforced
Invalid JWT header	Tampered Bearer token	✓ 401 Unauthorized

CONCLUSIONS

Key Security Finding: OTP rate limiting increases brute force attack difficulty from **15 minutes** to **5.1 years** — a 180,000x improvement — with zero impact on legitimate users who require only 1 attempt.

- No-registration OTP booking reduces friction for elderly and digitally underserved patients
- First platform to integrate NGO financial assistance within a clinic appointment system
- 4-provider AI fallback chain ensures near-100% chatbot uptime with no single point of failure
- Database-persistent prescriptions survive page refresh and are downloadable from the patient portal
- SUS usability score of **76/100** ("Good"), above 68 industry average; avg booking time ~95 seconds
- All 52 test cases pass; OWASP A01, A02, A05, A07 security controls fully implemented

FUTURE WORK

- AWS S3 cloud file storage to replace local Multer uploads for production scalability
- WebRTC telemedicine — in-platform video consultations between patient and doctor
- ABDM / ABHA Health ID integration — link patient records to India's national digital health system
- React Native mobile app and offline-first PWA support for rural low-connectivity users
- HL7 FHIR integration for hospital EHR system interoperability

REFERENCES

[1] Kumar & Singh (2021). Cloud-based clinic management for rural India. *J. Health Informatics Dev. Countries*, 15(2).
[2] Ometov et al. (2018). Multi-factor authentication: A survey. *Cryptography*, 2(1).
[3] Cascella et al. (2023). Evaluating ChatGPT in healthcare. *J. Medical Systems*, 47(1), 33.
[4] Altuwaijri & Ghilan (2022). Rate limiting as OTP brute force countermeasure. *Int. J. Network Security*, 24(3).
[5] OWASP (2021). OWASP Top Ten. Open Web Application Security Project.
[6] Grassi et al. (2017). NIST SP 800-63B: Digital Identity Guidelines. NIST.